Série d'exercices : PHP procédural

echo – chaînes – define – include – die – GET / POST – conversion – tableaux – dates – boutons radio – MySQLi

4ème SI – Lycée Rafèha – Enseignant : Aymen Essouyah

Sommaire

Partie 1 – echo, variables et chaînes (' et ") 2

Partie 2 – define, include et die 3

Partie 3 – GET et POST 4

Partie 4 – Conversion, conditions et boucles 5

Partie 5 – Tableaux et tableaux associatifs 6

Partie 6 – Les dates 8

Partie 7 – Formulaires avec boutons radio 10

Partie 8 – MySQLi procédural (base Dar Zitouna) 11

Annexe – Base de données Dar Zitouna 14

Consignes

- **PHP procédural uniquement** : utiliser des fonctions. Pour MySQL : `mysqli_connect`, `mysqli_query`, `mysqli_num_rows`, `mysqli_fetch_row`, `mysqli_fetch_array`, `mysqli_affected_rows`
- Les noms de fichiers et de champs de formulaire (`name`) sont **imposés** : ils permettent de comparer votre programme au jeu d'essai. Sauf indication contraire, le formulaire et son traitement sont dans **le même fichier**.
- Les programmes se testent avec un serveur (XAMPP / WAMP) : fichier dans `htdocs`, adresse `http://localhost/dossier/fichier.php`.
- Les exercices avec la base de données utilisent la base **Dar Zitouna** (annexe) et un fichier `config.php` à créer une fois et à inclure avec `require`.
- Dans tous les exercices, la date du jour supposée est le **samedi 14 novembre 2026** (jeux d'essai compris).
- Utiliser for, while, substr, `strpos`, `substr_count`, `array_keys`, `count`, `mktime`...
- Un cadre « Réponse » est à remplir sur la feuille ; « Fichier à produire » signifie : écrire le programme sur l'ordinateur.

Partie 1 – echo, variables et chaînes (' et ")

Exercice 1.1 – Prédire l'affichage (guillemets simples et doubles)

Écrire **exactement** ce que le navigateur affiche (une ligne par `<br>`).

```
<?php
$nom = "Rim";
$age = 17;
echo 'Nom : $nom<br>';
echo "Nom : $nom<br>";
echo "Dans un an : " . ($age + 1) . " ans<br>";
echo "Prix : \$5 ou 5 DT<br>";
echo 'L\'été<br>';
echo $nom . $age . "<br>";
echo $nom, $age + 3;
?>
```

Réponse :

Exercice 1.2 – Corriger les erreurs

Ce script contient **cinq erreurs** (syntaxe, guillemets, opérateur, variable). Les repérer, les expliquer, puis réécrire le script corrigé. Il doit afficher : le message de bienvenue, l'âge, un champ de saisie pré-rempli et le total.

```
<?php
nom = "Sami"
$age = "17";
echo "Bonjour $nom<br>";
echo "Vous avez " + $age + " ans<br>";
echo "<input type="text" name="nom" value="$nom"><br>";
echo 'Total : ' . $prix . ' DT';
?>
```

Réponse :

Exercice 1.3 – Écrire du HTML avec echo

Avec `$nom = "Sami";` et `$prix = 12.5;`, écrire **deux versions** de l'instruction `echo` qui produisent exactement la ligne HTML suivante : version A avec des guillemets simples pour PHP, version B avec des guillemets doubles pour PHP.

```
<a href="fiche.php?nom=Sami&prix=12.5" class="lien">Sami</a>
```

Réponse :

Exercice 1.4 – Fonctions sur les chaînes

Avec `$s = " dar zitouna ";`, écrire le code qui affiche, **chacun sur une ligne** :

a) la chaîne sans les espaces de début et de fin, entre crochets : `[dar zitouna]`

b) son nombre de caractères (après suppression des espaces)

c) chaque mot avec une majuscule, puis tout en majuscules

d) la chaîne où chaque `o` est remplacé par `0`

e) ses 3 premières lettres, puis la position du mot `zit`

f) le 2e mot (avec `strpos` et `substr`) et le nombre de mots (avec `substr_count`)

Réponse :

Partie 2 – define, include et die

Exercice 2.1 – Constantes et TVA

Écrire `tva.php` : définir avec `define` les constantes `TVA` (0.19), `DEVISE` ("DT") et `RESTAURANT` ("Dar Zitouna"). À partir du tableau associatif `$plats = ["Couscous" => 22, "Poisson" => 28, "Lablabi" => 9];` (prix hors taxe), afficher le nom du restaurant puis, pour chaque plat (parcourir les clés avec `array_keys` et une boucle `for`), le prix HT, la TVA et le prix TTC avec 2 décimales. Pourquoi `echo "TVA";` n'affiche-t-il pas 0.19 ?

Fichier à produire : tva.php

Affichage obtenu

```
Dar Zitouna
Couscous : HT 22.00 DT, TVA 4.18, TTC 26.18 DT
Poisson : HT 28.00 DT, TVA 5.32, TTC 33.32 DT
Lablabi : HT 9.00 DT, TVA 1.71, TTC 10.71 DT
```

Exercice 2.2 – Organiser un site avec include et require

Créer quatre fichiers : `config.php` (constantes `RESTAURANT` et `ANNEE`, valant 2026), `entete.php` (titre du restaurant et menu avec deux liens : Accueil, Contact), `pied.php` (copyright avec l'année) et `accueil.php` qui les assemble. Justifier le choix de `require` ou `include` pour chaque fichier.

Fichier à produire : accueil.php

Affichage obtenu

```
<html><body>
<h1>Dar Zitouna</h1>
<a href="accueil.php">Accueil</a> | <a href="contact.php">Contact</a>
<hr>
<p>Bienvenue dans notre restaurant.</p>
<hr>
<p>&copy; 2026 Dar Zitouna</p>
</body></html>
```

Exercice 2.3 – Arrêter le script avec die

Écrire `fiche.php` qui reçoit un numéro `num` dans l'adresse et utilise le tableau `$serveurs = [1 => "Sami", 2 => "Rim", 3 => "Karim", 4 => "Nour"];`. Utiliser `die` pour : numéro absent (« Numéro manquant »), numéro non numérique (« Numéro invalide »), numéro absent du tableau (« Serveur inconnu »). Sinon afficher « Serveur n°3 : Karim ».

Fichier à produire : fiche.php

Jeu d'essai « fiche.php?num=3 » – données envoyées : num = 3

```
Serveur n°3 : Karim
```

Partie 3 – GET et POST

Exercice 3.1 – Vrai ou faux ?

Répondre par **V** ou **F** et corriger les affirmations fausses.

a) `$_POST["nom"]` lit le champ dont l'attribut `id` vaut `nom`.

b) Avec `method="get"`, les données du formulaire apparaissent dans l'adresse de la page.

c) Un formulaire de connexion doit utiliser la méthode GET.

d) Un bouton radio non coché est envoyé avec une valeur vide.

e) Les valeurs de `$_POST` sont toujours de type chaîne.

f) `isset($_POST["valider"])` permet de savoir si le formulaire a été soumis (le bouton a un `name`).

g) On peut lire `$_GET["x"]` sans le tester : au pire la valeur est vide.

h) `include "a.php";` arrête le script si `a.php` est absent.

Réponse :

Exercice 3.2 – Inscription d'un serveur (POST)

Écrire `inscription.php` (formulaire et traitement dans le même fichier, méthode POST) avec : `nom` et `prenom` (texte), `salle` (liste déroulante : Terrasse, Salon) et le bouton `inscrire`. Après envoi : si un des deux champs est vide (utiliser `trim` et `empty`), afficher « Nom et prénom obligatoires. » ; sinon afficher « Bienvenue Prénom NOM, vous travaillerez en salle X. » (prénom avec majuscule initiale, nom en majuscules).

Fichier à produire : inscription.php

Jeu d'essai « champs remplis » – données envoyées : nom = ben salah, prenom = AYA, salle = Salon, inscrire = ok

```
Bienvenue Aya BEN SALAH, vous travaillerez en salle Salon.
```

Exercice 3.3 – Menu et fiche d'un plat (liens GET et include)

Écrire trois fichiers : `donnees.php` (tableau `$menu` : 3 plats avec clés numériques 1, 2, 3 et, pour chacun, `nom`, `prix`, `categorie` : Salade tunisienne 6 DT Entrée, Couscous à l'agneau 22 DT Plat, Makroudh 5 DT Dessert) ; `menu.php` qui inclut `donnees.php` et affiche chaque plat (boucle `for`) sous forme de **lien** `plat.php?num=...` ; `plat.php` qui affiche la fiche du plat demandé (nom, catégorie, prix) ou « Plat introuvable » (avec `die`) si le numéro est absent ou inconnu.

Fichier à produire : menu.php, plat.php, donnees.php

Jeu d'essai « menu.php »

```
<a href='plat.php?num=1'>Salade tunisienne</a><br><a href='plat.php?num=2'>Couscous à
l'agneau</a><br><a href='plat.php?num=3'>Makroudh</a><br>
```

Jeu d'essai « clic sur le 2e lien » – données envoyées : num = 2

```
Couscous à l'agneau
Catégorie : Plat
Prix : 22 DT
```

Exercice 3.4 – Table de multiplication (GET)

Écrire `table.php` avec un formulaire **GET** (champ `n`, bouton `go`). Après envoi, si `n` n'est pas un nombre, afficher « Saisir un nombre. » ; sinon convertir `n` en entier et afficher la table de multiplication de `n` de 1 à 10 (`7 x 3 = 21`). Observer l'adresse de la page après l'envoi.

Fichier à produire : table.php

Jeu d'essai « n = 7 » – données envoyées : n = 7, go = Afficher

```
Table de 7 :
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Partie 4 – Conversion, conditions et boucles

Exercice 4.1 – Prédire les types et les conversions

Écrire ce qu'affiche chaque `var_dump` (type **et** valeur).

```php
<?php
var_dump("7" + "3");
var_dump("7" . "3");
var_dump((int)"15 DT");
var_dump((int)4.9);
var_dump((bool)"0");
var_dump((bool)"false");
var_dump("10" == 10);
var_dump("10" === 10);
var_dump(is_numeric("1e3"));
?>
```

Réponse :

Exercice 4.2 – Indice de masse corporelle (POST)

Écrire `imc.php` : formulaire POST (`poids` en kg, `taille` en m, bouton `calculer`). Vérifier avec `is_numeric` que les deux valeurs sont des nombres strictement positifs (sinon « Saisie incorrecte. »), convertir en réel, calculer `IMC = poids / (taille × taille)` arrondi à 1 décimale et afficher l'interprétation : moins de 18.5 « maigreur », moins de 25 « poids normal », moins de 30 « surpoids », sinon « obésité ».

Fichier à produire : imc.php

Jeu d'essai « poids 70, taille 1.75 » – données envoyées : poids = 70, taille = 1.75, calculer = ok

```
IMC = 22.9 : poids normal
```

Exercice 4.3 – Boucles

Écrire `boucles.php` qui affiche, avec les boucles demandées : (a) avec `for`, un triangle de 5 lignes d'étoiles (1 étoile sur la 1re ligne, 2 sur la 2e…, utiliser `str_repeat`) ; (b) avec `while`, la somme des entiers de 1 à 100 ; (c) avec `for`, les carrés des nombres de 1 à 10 sur une seule ligne, séparés par ` - ` (sans ` - ` après le dernier).

Fichier à produire : boucles.php

Affichage obtenu

```
*
**
***
****
*****
Somme : 5050
1 - 4 - 9 - 16 - 25 - 36 - 49 - 64 - 81 - 100
```

Exercice 4.4 – Listes déroulantes générées par des boucles

Écrire `date_naissance.php` qui affiche trois listes déroulantes générées **uniquement avec des boucles** : `jour` (1 à 31), `mois` (les noms des mois en français, stockés dans un tableau ; la `value` est le numéro du mois), `annee` (de l'année actuelle jusqu'à 1990, en décroissant ; l'année actuelle est présélectionnée avec `selected`).

Fichier à produire : date_naissance.php

Partie 5 – Tableaux et tableaux associatifs

Exercice 5.1 – Prédire l'affichage

Écrire ce que le navigateur affiche.

```
<?php
$s = ["nom" => "Rim", "age" => 17, "salle" => "Salon"];
$s["age"] = $s["age"] + 1;
$s["tel"] = "71 000 000";
unset($s["salle"]);
$cles = array_keys($s);
for ($i = 0; $i < count($cles); $i++) {
    echo $cles[$i] . "=" . $s[$cles[$i]] . ";";
}
echo count($s);
echo isset($s["salle"]) ? " oui" : " non";
$notes = [12, 8, 15];
$notes[] = 10;
sort($notes);
echo "<br>Notes :";
for ($i = 0; $i < count($notes); $i++) {
    echo " " . $notes[$i];
}
echo " | max " . max($notes) . " | 2e : " . $notes[1];
?>
```

Réponse :

Exercice 5.2 – Statistiques sur un tableau indexé

Écrire `notes.php`. Avec `$notes = [12, 8, 15, 9.5, 17, 10];`, afficher : le nombre de notes, la moyenne (2 décimales), la meilleure et la plus faible note, le nombre de notes supérieures ou égales à 10 (boucle `for`), la liste triée par ordre **décroissant** (`rsort`, puis boucle `for` avec des virgules entre les notes, sans virgule après la dernière), puis « Admis » si la moyenne est supérieure ou égale à 10, « Ajourné » sinon.

Fichier à produire : notes.php

Affichage obtenu

```
Nombre : 6
Moyenne : 11.92
Max : 17 - Min : 8
Notes >= 10 : 4
Tri décroissant : 17, 15, 12, 10, 9.5, 8
Admis
```

Exercice 5.3 – Tableau de tableaux : bulletin d'une classe

Écrire `classe.php` avec le tableau `$eleves` suivant : Aya 14.5, Sami 9, Rim 16, Karim 11.5 (chaque élève est un tableau associatif `nom` / `moyenne`). Avec une boucle `for`, afficher un tableau HTML (nom, moyenne, mention : « Bien » à partir de 14, « Passable » à partir de 10, sinon « Insuffisant »), puis la moyenne de la classe, le nom du meilleur élève et le nombre d'élèves admis (moyenne ≥ 10).

Fichier à produire : classe.php

Affichage obtenu

```
Aya | 14.5 | Bien
Sami | 9 | Insuffisant
Rim | 16 | Bien
Karim | 11.5 | Passable
Moyenne de la classe : 12.75
Meilleur élève : Rim
Admis : 3
```

Exercice 5.4 – Compter les occurrences (tableau associatif)

Avec le tableau `$villes = ["tunis", "sfax", "tunis", "sousse", "sfax", "tunis"];`, construire un tableau associatif `$compte` (clé = ville, valeur = nombre d'occurrences) avec une boucle `for` et `isset`. Afficher le résultat trié par nom de ville (`ksort`, puis `array_keys` et une boucle `for`) avec une majuscule initiale : « Sfax : 2 », « Sousse : 1 », « Tunis : 3 ».

Fichier à produire : villes.php

Affichage obtenu

```
Sfax : 2
Sousse : 1
Tunis : 3
```

Partie 6 – Les dates

Exercice 6.1 – Formats de date

On pose `$t = mktime(9, 5, 30, 12, 25, 2026);` (vendredi 25 décembre 2026, 9 h 05 min 30 s). **(1)** Écrire le format `date()` qui affiche : (a) `2026-12-25` ; (b) `25-12-26` ; (c) `9h05` ; (d) `Fri, 25 Dec` ; (e) `09:05:30`. **(2)** Prédire l'affichage du script ci-dessous.

```php
<?php
$t = mktime(9, 5, 30, 12, 25, 2026);
echo date("d/m/Y", $t) . "<br>";
echo date("D j M y", $t) . "<br>";
echo date("H:m:s", $t) . "<br>";
echo date("g:i a", $t) . "<br>";
echo date("N w z", $t) . "<br>";
echo date("t L", $t);
?>
```

Réponse :

Exercice 6.2 – La date du jour en français

Écrire `aujourdhui.php` qui affiche la date du jour sous la forme « Aujourd'hui : samedi 14 novembre 2026 » (jour de la semaine et mois **en français**, à l'aide de deux tableaux et de `date("w")`, `date("j")`, `date("n")`, `date("Y")`).

Fichier à produire : aujourdhui.php

Jeu d'essai « exécuté le 14/11/2026 »

```
Aujourd'hui : samedi 14 novembre 2026
```

Exercice 6.3 – Calcul de l'âge (formulaire avec bouton radio)

Écrire `age.php` (POST) : champ `naissance` (`type="date"`), deux boutons radio `titre` (Monsieur / Madame) et le bouton `calculer`. Si la date ou la civilité manque, afficher « Choisissez une civilité et une date de naissance. ». Sinon afficher « Monsieur, vous avez 18 ans (majeur). » (ou « (mineur) ») puis « Prochain anniversaire dans N jour(s). ». La date arrive au format `AAAA-MM-JJ` : extraire l'année, le mois et le jour avec `substr` . Attention : l'âge est la différence des années, **moins 1** si l'anniversaire de cette année n'est pas encore passé (comparer `date("md")` avec le mois et le jour de naissance collés).

Fichier à produire : age.php

Jeu d'essai « au 14/11/2026 : Monsieur, né le 2008-05-12 » – données envoyées : titre = Monsieur, naissance = 2008-05-12, calculer = ok

```
Monsieur, vous avez 18 ans (majeur).
Prochain anniversaire dans 179 jour(s).
```

Exercice 6.4 – Différence entre deux dates

Écrire `difference.php` (POST) : deux champs `d1` et `d2` de type date et le bouton `calculer`. Si une date manque : « Deux dates sont nécessaires. » ; si `d2` est avant `d1` : « La 2e date doit être postérieure à la 1re. » ; sinon afficher le nombre de **jours** entre les deux dates, puis le même écart en « semaines et jours » (par exemple `42 semaine(s) et 1 jour(s)`).

Fichier à produire : difference.php

Jeu d'essai « 2026-03-05 et 2026-12-25 » – données envoyées : d1 = 2026-03-05, d2 = 2026-12-25, calculer = ok

```
295 jour(s) d'écart.
Soit 42 semaine(s) et 1 jour(s).
```

Exercice 6.5 – Date de livraison

Écrire `livraison.php` (POST) : champs `date` (date de commande, type date), `delai` (nombre de jours, type number) et bouton `calculer`. La date de livraison est la date de commande **plus** `delai` jours ; si elle tombe un **dimanche**, la livraison est reportée au lundi. Afficher « Livraison le lundi 16/03/2026. » (jour de la semaine en français, date au format `jj/mm/aaaa`).

Fichier à produire : livraison.php

Jeu d'essai « 2026-03-05 + 10 jours (le 15/03 est un dimanche) » – données envoyées : date = 2026-03-05, delai = 10, calculer = ok

```
Livraison le lundi 16/03/2026.
```

Exercice 6.6 – Calendrier d'un mois

Écrire `mois.php` (POST) : liste déroulante `mois` (noms français, `value` de 1 à 12), champ `annee` et bouton `voir`. Si l'année n'est pas un nombre, afficher « Année invalide. » ; sinon afficher : le nombre de jours du mois, le jour de la semaine du 1er du mois (en français) et si l'année est bissextile. Exemple : « Mars 2026 : 31 jours. Le 1er est un dimanche. Année bissextile : non. »

Fichier à produire : mois.php

Jeu d'essai « mois = 3, annee = 2026 » – données envoyées : mois = 3, annee = 2026, voir = ok

```
Mars 2026 : 31 jours.
Le 1er est un dimanche.
Année bissextile : non
```

Exercice 6.7 – Changer le format d'une date

Sans formulaire . (a) Convertir `$fr = "25/12/2026";` en `2026-12-25` de **deux** façons : en extrayant le jour, le mois et l'année avec `substr` puis en les recollant ; avec `mktime` puis `date`. (b) Convertir `$sql = "2026-03-05";` en `05/03/2026` avec `date` et `strtotime`. (c) Pourquoi ne peut-on pas écrire `strtotime("25/12/2026")` ? (d) Vérifier avec `checkdate` si `"31/02/2026"` existe (attention à l'ordre des arguments).

Fichier à produire : formats_date.php

Partie 7 – Formulaires avec boutons radio

Exercice 7.1 – Civilité (boutons radio)

Écrire `bonjour.php` (POST) : champs `prenom` et `nom` (texte), boutons radio `civilite` (Monsieur / Madame) et bouton `envoyer`. Après envoi, si aucune civilité n'est cochée ou si un champ est vide : « Veuillez tout remplir. » ; sinon « Bonjour Madame Prénom NOM ». Quelle valeur contient `$_POST["civilite"]` ? Que se passe-t-il si aucun bouton n'est coché ?

Fichier à produire : bonjour.php

Jeu d'essai « civilité cochée » – données envoyées : civilite = Madame, prenom = aya, nom = Ben Salah, envoyer = ok

```
Bonjour Madame Aya BEN SALAH
```

Exercice 7.2 – Calculatrice à boutons radio

Écrire `calcul.php` (POST) : champs `a` et `b`, cinq boutons radio `op` (`+`, `-`, `*`, `/`, `%` pour le reste), bouton `calculer`. Vérifier les saisies avec `is_numeric` et la présence d'une opération (sinon « Saisie incorrecte. »). Utiliser `switch`. Pour `/` et `%`, afficher « Division par zéro impossible. » si `b` vaut 0. Les valeurs saisies et l'opération choisie doivent **rester affichées** après l'envoi (`checked` conditionnel). Exemple : « 17 % 5 = 2 ».

Fichier à produire : calcul.php

Jeu d'essai « 17 % 5 » – données envoyées : a = 17, b = 5, op = %, calculer = =

```
17 % 5 = 2
```

Exercice 7.3 – QCM de 5 questions (boutons radio)

Écrire `qcm.php` (POST) : le formulaire est **généré** par une boucle à partir d'un tableau `$questions` (chaque question : énoncé, 3 choix, indice de la bonne réponse). Les boutons radio de la question *i* s'appellent `q0`, `q1`… et ont pour `value` l'indice du choix. Après `corriger`, afficher « Score : N / 5 » et une mention : 5 « Excellent », 3 ou 4 « Bien », sinon « À revoir ». Une question sans réponse compte 0. Proposer vos 5 questions sur GET/POST, define, include, die, mysqli.

Fichier à produire : qcm.php

Jeu d'essai « 4 bonnes réponses, la 5e sans réponse » – données envoyées : q0 = 0, q1 = 1, q2 = 2, q3 = 1, corriger = ok

```
Score : 4 / 5
Bien
```

Exercice 7.4 – Convertisseur de monnaie (radio et constantes)

Écrire `convertisseur.php` (POST) : champ `montant`, trois boutons radio `sens` (`dt_eur` : DT → EUR, `eur_dt` : EUR → DT, `dt_usd` : DT → USD), bouton `convertir`. Définir avec `define` les constantes `DT_EUR` (0.30) et `DT_USD` (0.32) (taux **fictifs**). Afficher le résultat avec 2 décimales, par exemple « 100 → 30.00 EUR ». Le montant et le choix doivent rester affichés après l'envoi.

Fichier à produire : convertisseur.php

Jeu d'essai « 100 DT → EUR » – données envoyées : montant = 100, sens = dt_eur, convertir = ok

```
100 → 30.00 EUR
```

Exercice 7.5 – Commande au restaurant (radio, quantité et remise)

Écrire `commande.php` (POST) avec `$menu = [1 => ["Salade tunisienne", 6], 2 => ["Couscous à l'agneau", 22], 3 => ["Makroudh", 5]];`. Le formulaire contient : des boutons radio `plat` **générés par une boucle** `for` (value = clé du tableau), un champ `qte` (nombre, 1 par défaut), deux boutons radio `mode` (`place` coché par défaut, `emporter`) et le bouton `commander`. Après envoi : si un champ est incorrect (`qte` non numérique ou ≤ 0, plat non choisi) « Veuillez remplir tout le formulaire. » ; sinon afficher la ligne « 3 x Couscous à l'agneau = 66 DT », puis, pour `emporter`, « Remise à emporter (10 %) » et enfin « Total à payer : 59.40 DT ».

Fichier à produire : commande.php

Jeu d'essai « Couscous, 3, à emporter » – données envoyées : plat = 2, qte = 3, mode = emporter, commander = ok

```
3 x Couscous à l'agneau = 66 DT
Remise à emporter (10 %)
Total à payer : 59.40 DT
```

Partie 8 – MySQLi procédural (base Dar Zitouna)

Exercice 8.1 – Prédire les résultats (num_rows, fetch, affected_rows)

Avec les tables de l'annexe (`SERVEUR`, `COMMANDE`), écrire ce que le navigateur affiche. La base est réinitialisée avant le script.

```php
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$res = mysqli_query($cnx, "SELECT nom, salle FROM SERVEUR WHERE salle = 'Salon'");
echo mysqli_num_rows($res) . "<br>";
$l = mysqli_fetch_row($res);
echo $l[0] . "<br>";
$l = mysqli_fetch_array($res);
echo $l["nom"] . " " . $l["salle"] . "<br>";
var_dump(mysqli_fetch_row($res));
$sql = "SELECT plat, prix FROM COMMANDE WHERE categorie = 'Dessert' ORDER BY prix";
$res = mysqli_query($cnx, $sql);
$l = mysqli_fetch_array($res);
echo $l["plat"] . " " . $l["prix"] . "<br>";
mysqli_query($cnx, "UPDATE COMMANDE SET prix = prix + 1 WHERE categorie = 'Dessert'");
echo mysqli_affected_rows($cnx) . "<br>";
mysqli_query($cnx, "UPDATE COMMANDE SET qte = 2 WHERE num = 11");
echo mysqli_affected_rows($cnx) . "<br>";
mysqli_query($cnx, "DELETE FROM COMMANDE WHERE categorie = 'Boisson'");
echo mysqli_affected_rows($cnx);
?>
```

Réponse :

Exercice 8.2 – Liste des serveurs

Écrire `serveurs.php` : se connecter avec `require "config.php"` et `or die(...)`, puis afficher dans un tableau HTML le numéro, le nom, la salle et la date d'embauche au format `jj/mm/aaaa` de chaque serveur (`mysqli_fetch_row`), précédé de « N serveur(s) » (`mysqli_num_rows`).

Fichier à produire : serveurs.php

Affichage obtenu

```
4 serveur(s)
1 | Sami | Terrasse | 01/01/2022
2 | Rim | Salon | 01/09/2023
3 | Karim | Terrasse | 15/02/2019
4 | Nour | Salon | 10/01/2024
```

Exercice 8.3 – Serveurs d'une salle (boutons radio générés par la base)

Écrire `salles.php` (POST). Les boutons radio `salle` sont **générés** par une requête `SELECT DISTINCT salle FROM SERVEUR` (avec `mysqli_fetch_row`). Après envoi (`afficher`), afficher le nombre de serveurs de la salle choisie, puis, pour chacun (avec `mysqli_fetch_array`), son nom et le nombre de commandes qu'il a prises. Utiliser la requête `SELECT nom, (SELECT COUNT(*) FROM COMMANDE WHERE COMMANDE.numserveur = SERVEUR.num) AS nb FROM SERVEUR WHERE salle = '$salle'`. Sans salle choisie : « Choisissez une salle. »

Fichier à produire : salles.php

Jeu d'essai « salle = Terrasse » – données envoyées : salle = Terrasse, afficher = ok

```
2 serveur(s) en Terrasse
Sami : 4 commande(s)
Karim : 4 commande(s)
```

Exercice 8.4 – Ajouter un serveur (INSERT et affected_rows)

Écrire `ajout_serveur.php` (POST) : champs `num`, `nom`, boutons radio `salle` (Terrasse / Salon), champ `date` (type date) et bouton `ajouter`. Après envoi, si un champ manque ou si `num` n'est pas numérique : « Veuillez remplir tous les champs. » ; sinon exécuter un `INSERT` ; si `mysqli_query` renvoie `true` et que `mysqli_affected_rows` vaut 1 : « Serveur ajouté. » ; sinon « Échec : ce numéro existe déjà. ». Terminer en affichant le nombre total de serveurs (`mysqli_num_rows`). On suppose que le nom ne contient pas d'apostrophe.

Fichier à produire : ajout_serveur.php

Jeu d'essai « num 5, Aya, Salon, 2025-09-01 » – données envoyées : num = 5, nom = Aya, salle = Salon, date = 2025-09-01, ajouter = ok

```
Serveur ajouté.
Total : 5 serveur(s)
```

Exercice 8.5 – Modifier ou supprimer une commande (radio + UPDATE / DELETE)

Écrire `gestion.php` (POST) : champ `num` (numéro de commande), champ `prix` (nouveau prix), boutons radio `action` (`modifier` / `supprimer`) et bouton `valider`. Pour `modifier` : exécuter l'`UPDATE` du prix ; `affected_rows` = 1 → « Prix modifié. », 0 → « Aucune modification (numéro inconnu ou même prix). ». Pour `supprimer` : exécuter le `DELETE` ; 1 → « Commande supprimée. », 0 → « Numéro inconnu. ». Si aucune action n'est choisie ou si `num` n'est pas numérique : « Saisie incorrecte. ».

Fichier à produire : gestion.php

Jeu d'essai « modifier la commande 1, prix 7 » – données envoyées : num = 1, prix = 7, action = modifier, valider = ok

```
Prix modifié.
```

Exercice 8.6 – Ancienneté des serveurs (dates et base de données)

Écrire `anciennete.php` : pour chaque serveur (`mysqli_fetch_array`), afficher « Nom : N an(s) » où N est l'ancienneté en **années entières** au **1er janvier 2027**, calculée en PHP à partir de `date_embauche` (extraire l'année avec `substr`), puis « (plus de 5 ans) » si N > 5, sinon « (5 ans ou moins) ».

Fichier à produire : anciennete.php

Affichage obtenu

```
Sami : 5 an(s) (5 ans ou moins)
Rim : 3 an(s) (5 ans ou moins)
Karim : 7 an(s) (plus de 5 ans)
Nour : 2 an(s) (5 ans ou moins)
```

Exercice 8.7 – Commandes d'un mois (liste déroulante et dates SQL)

Écrire `commandes_mois.php` (POST) : liste déroulante `mois` (noms français générés par une boucle, `value` de 1 à 12), liste `annee` (2026, 2027) et bouton `voir`. Afficher les commandes dont `date_cmd` est dans le mois choisi (`MONTH(date_cmd) = $mois AND YEAR(date_cmd) = $annee`) dans un tableau (plat, quantité, date au format `jj/mm/aaaa`) puis le chiffre d'affaires du mois (`prix × qte`). S'il n'y a aucune commande : « Aucune commande ce mois-ci. » (`mysqli_num_rows`).

Fichier à produire : commandes_mois.php

Jeu d'essai « mars 2026 » – données envoyées : mois = 3, annee = 2026, voir = ok

```
Salade tunisienne | 2 | 15/03/2026
Brik à l'oeuf | 5 | 15/03/2026
Couscous à l'agneau | 3 | 22/03/2026
Poisson grillé | 2 | 28/03/2026
Chiffre d'affaires de Mars 2026 : 154 DT
```

Annexe – Base de données Dar Zitouna

Script de création et d'insertion (à exécuter dans phpMyAdmin après avoir créé la base `darzitouna`). C'est la base de la série 8, avec deux colonnes de dates en plus.

```
CREATE TABLE SERVEUR (
 num INT PRIMARY KEY,
 nom VARCHAR(20),
 salle VARCHAR(20),
 date_embauche DATE
);
CREATE TABLE COMMANDE (
 num INT PRIMARY KEY,
 plat VARCHAR(30),
 categorie VARCHAR(15),
 prix DECIMAL(5,2),
 qte INT,
 numserveur INT,
 date_cmd DATE,
 FOREIGN KEY (numserveur) REFERENCES SERVEUR(num)
);
INSERT INTO SERVEUR VALUES
 (1,'Sami','Terrasse','2022-01-01'), (2,'Rim','Salon','2023-09-01'),
 (3,'Karim','Terrasse','2019-02-15'), (4,'Nour','Salon','2024-01-10');
INSERT INTO COMMANDE VALUES
 (1,'Salade tunisienne','Entrée',6,2,1,'2026-03-15'),
 (2,'Brik à l''oeuf','Entrée',4,5,2,'2026-03-15'),
 (3,'Couscous à l''agneau','Plat',22,3,1,'2026-03-22'),
 (4,'Poisson grillé','Plat',28,2,3,'2026-03-28'),
 (5,'Ojja merguez','Plat',18,4,2,'2026-04-15'),
 (6,'Makroudh','Dessert',5,6,3,'2026-06-10'),
 (7,'Assidat zgougou','Dessert',7,3,1,'2026-11-15'),
 (8,'Thé à la menthe','Boisson',3,8,2,'2027-01-15'),
 (9,'Jus d''orange','Boisson',4,5,3,'2027-02-20'),
 (10,'Brik au thon','Entrée',4,4,1,'2027-03-15'),
 (11,'Lablabi','Plat',9,2,2,'2027-04-08'),
 (12,'Eau minérale','Boisson',2,10,3,'2027-04-14');
```

Fichier config.php (à créer et à inclure avec require)

```
<?php
define("HOTE", "localhost");
define("UTIL", "root");
define("MDP", "");
define("BASE", "darzitouna");
mysqli_report(MYSQLI_REPORT_OFF);
?>
```

`mysqli_report(MYSQLI_REPORT_OFF);` : depuis PHP 8.1, sans cette ligne une erreur MySQL lève une exception au lieu de renvoyer `false` et `or die(...)` n'est jamais exécuté. Adapter `UTIL` et `MDP` à votre serveur (en général `root` et un mot de passe vide avec XAMPP).

Enseignant : Aymen Essouyah – Lycée Rafèha – 4ème SI